

Сначала напишем модуль task3. Для этого возьмём модуль task2 из предыдущего отчёта и дополним его. Добавим функции test_proc_read и test_proc_write для чтения и записи в proc.

```
78 static ssize_t test_proc_read(struct file* filp, char __user* buff,
79                               size_t count, loff_t* off) {
80     size_t rc;
81     rc = simple_read_from_buffer(buff, count, off, test_string, 15);
82
83     return rc;
84 }
85
86 static ssize_t test_proc_write(struct file* filp, const char __user* buff,
87                                size_t count, loff_t* off) {
88     size_t rc = 0;
89     if (count > 15) {
90         return -EINVAL;
91     }
92
93     rc = simple_write_to_buffer(test_string, 15, off, buff, count);
94     return rc;
95 }
96
```

Рисунок 1 — функции test_proc_read и test_proc_write

Добавим функции string_show и string_store для чтения и записи в sys.

```
97 static ssize_t string_show(struct kobject* kobj, struct kobj_attribute* attr,
98                             char* buff) {
99     size_t rc = 0;
100     memcpy(buff, test_string, 15);
101     rc = strlen(test_string);
102     return rc;
103 }
104
105 static ssize_t string_store(struct kobject* kobj, struct kobj_attribute* attr,
106                             char const* buff, size_t count) {
107     size_t rc = 0;
108     memcpy(test_string, buff, count);
109     rc = strlen(buff);
110     return rc;
111 }
112
```

Рисунок 2 — функции string_show string_store

Затем зададим операции для proc_ops и атрибуты для kobject.

```
37 static const struct proc_ops pops = {
38     .proc_read = test_proc_read,
39     .proc_write = test_proc_write
40 };
41
42 static struct kobj_attribute string_attribute =
43     __ATTR(test_string, 0664, string_show, string_store);
44
45 static struct attribute* attrs[] = {
46     &string_attribute.attr,
47     NULL
48 };
49
50 static struct attribute_group attr_group = {
51     .attrs = attrs,
52 };
53
```

Рисунок 3 — операции для proc_ops и атрибуты для kobject

Изменим функции `init_module` и `cleanup_module` (функции инициализации и очистки модуля).

```
113 int init_module(void) {
114     int retval = 0;
115     pr_info("Module is loaded\n");
116     rwlock_init(&lock);
117     major = register_chrdev(major, "test", &fops);
118
119     if (major < 0) {
120         return major;
121     }
122     pr_info("Major = %d\n", major);
123
124     test = proc_create("test", 0666, NULL, &pops);
125
126     test_kobj = kobject_create_and_add("kobject_test", kernel_kobj);
127     if (!test_kobj) {
128         return -ENOMEM;
129     }
130
131     retval = sysfs_create_group(test_kobj, &attr_group);
132     if (retval) {
133         kobject_put(test_kobj);
134         return retval;
135     }
136
137     return 0;
138 }
139
140 void cleanup_module(void) {
141     unregister_chrdev(major, "test");
142     proc_remove(test);
143     kobject_put(test_kobj);
144     pr_info("Module is cleaned\n");
145 }
```

Рисунок 4 — функции `init_module` и `cleanup_module`

Теперь соберём модуль, загрузим его и выведем последние сообщения ядра. Убедимся, что модуль загрузился успешно.

```
shrek@hp-laptop:~/linux_kernel/modules$ make
make -C /lib/modules/6.8.0-117-generic/build M=/home/shrek/linux_kernel/modules modules
make[1]: вход в каталог «/usr/src/linux-headers-6.8.0-117-generic»
warning: the compiler differs from the one used to build the kernel
The kernel was built by: x86_64-linux-gnu-gcc-13 (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
You are using:          gcc-13 (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
CC [M] /home/shrek/linux_kernel/modules/task3.o
MODPOST /home/shrek/linux_kernel/modules/Module.symvers
CC [M] /home/shrek/linux_kernel/modules/task3.mod.o
LD [M] /home/shrek/linux_kernel/modules/task3.ko
BTF [M] /home/shrek/linux_kernel/modules/task3.ko
Skipping BTF generation for /home/shrek/linux_kernel/modules/task3.ko due to unavailability of vmlinux
make[1]: выход из каталога «/usr/src/linux-headers-6.8.0-117-generic»
shrek@hp-laptop:~/linux_kernel/modules$ sudo insmod task3.ko
[sudo] пароль для shrek:
shrek@hp-laptop:~/linux_kernel/modules$ dmesg | tail -n2
[10762.978832] Module is loaded
[10762.978842] Major = 237
shrek@hp-laptop:~/linux_kernel/modules$
```

Рисунок 5 — сборка и загрузка модуля `task3`

Если перейти в директорию /proc, то можно увидеть, что модуль task3 создал файл test.

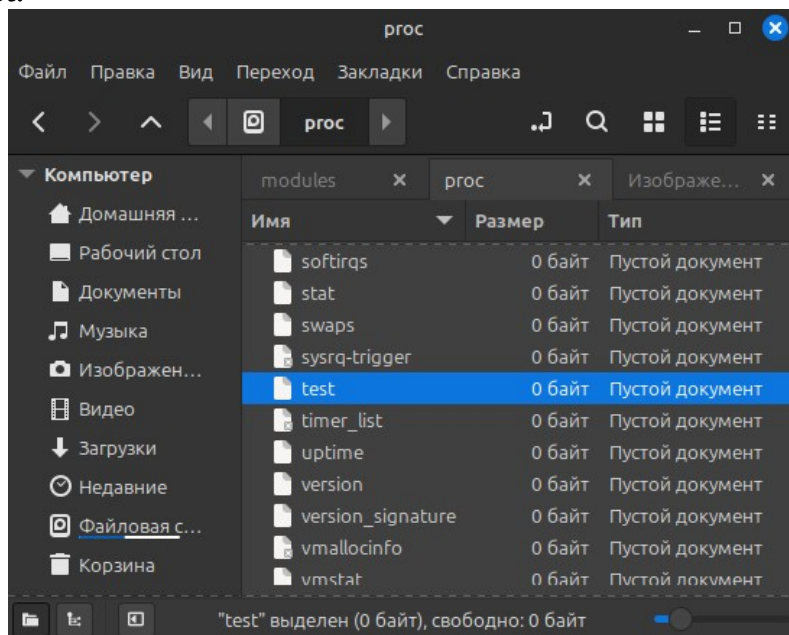


Рисунок 6 — содержимое директории /proc

Выводим содержимое переменной test_string и перезапишем её, обращаясь к файлу test.

```
shrek@hp-laptop:/proc$ cat test
Hello!
shrek@hp-laptop:/proc$ echo "ABCDEFGG" > test
shrek@hp-laptop:/proc$ cat test
ABCDEFGG
shrek@hp-laptop:/proc$
```

Рисунок 7 — вывод и изменение переменной test_string через файл test

Если перейти в директорию /sys/kernel, то можно увидеть, что модуль task3 создал папку kobject_test и файл test_string в этой папке.

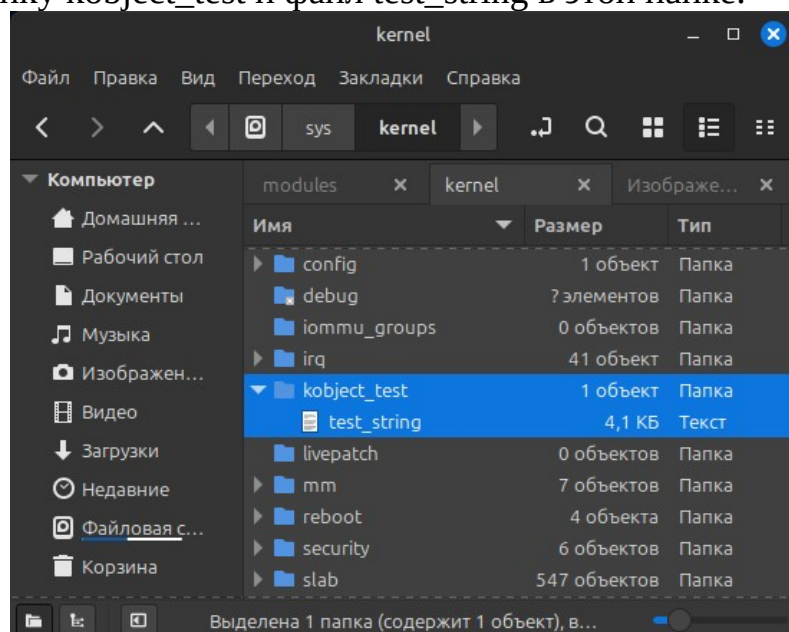


Рисунок 8 — содержимое директории /sys/kernel

Выводим содержимое переменной `test_string` и перезапишем её, обращаясь к файлу `test_string`. Содержимое переменной изменилось после предыдущего обращения, так как во время выполнения записи или чтения файлов `test` и `test_string`, вызываются функции, обращающиеся к одной и той же переменной.

```
shrek@hp-laptop:/sys/kernel$ cd kobject_test/
shrek@hp-laptop:/sys/kernel/kobject_test$ cat test_string
ABCDEFGH
shrek@hp-laptop:/sys/kernel/kobject_test$ echo "qwaqwaqwa" > test_string
bash: test_string: Отказано в доступе
shrek@hp-laptop:/sys/kernel/kobject_test$ echo "qwaqwaqwa" | sudo tee test_string > /dev/null
[sudo] пароль для shrek:
shrek@hp-laptop:/sys/kernel/kobject_test$ cat test_string
qwaqwaqwa
shrek@hp-laptop:/sys/kernel/kobject_test$
```

Рисунок 9 — вывод и изменение переменной `test_string` через файл `test_string`

Теперь можно выгрузить модуль `task3` и вывести последние сообщения ядра.

```
shrek@hp-laptop:~/linux_kernel/modules$ sudo rmmod task3
shrek@hp-laptop:~/linux_kernel/modules$ dmesg | tail -n2
[10762.978842] Major = 237
[11287.155179] Module is cleaned
```

Рисунок 10 — очистка модуля `task3`

Если зайти в директорию `/proc` или `/sys/kernel`, то можно убедиться, что ранее созданные папки и файлы были автоматически удалены.

```
shrek@hp-laptop:/proc$ ls | grep test
shrek@hp-laptop:/proc$ cd ..
shrek@hp-laptop:/$ cd sys
shrek@hp-laptop:/sys$ cd kernel/
shrek@hp-laptop:/sys/kernel$ ls
address_bits      fscaps            notes             software_nodes
boot_params       iommu_groups      oops_count        tracing
btf               irq               profiling          uevent_helper
cgroup            kexec_crash_loaded rcu_expedited      uevent_seqnum
config            kexec_crash_size  rcu_normal         vmcoreinfo
cpu_byteorder     kexec_loaded      reboot             warn_count
crash_elfcorehdr_size livepatch          security
debug             mm                slab
shrek@hp-laptop:/sys/kernel$ ls | grep kobject_test
shrek@hp-laptop:/sys/kernel$
```

Рисунок 11 — содержимое директорий `/proc` и `/sys/kernel`